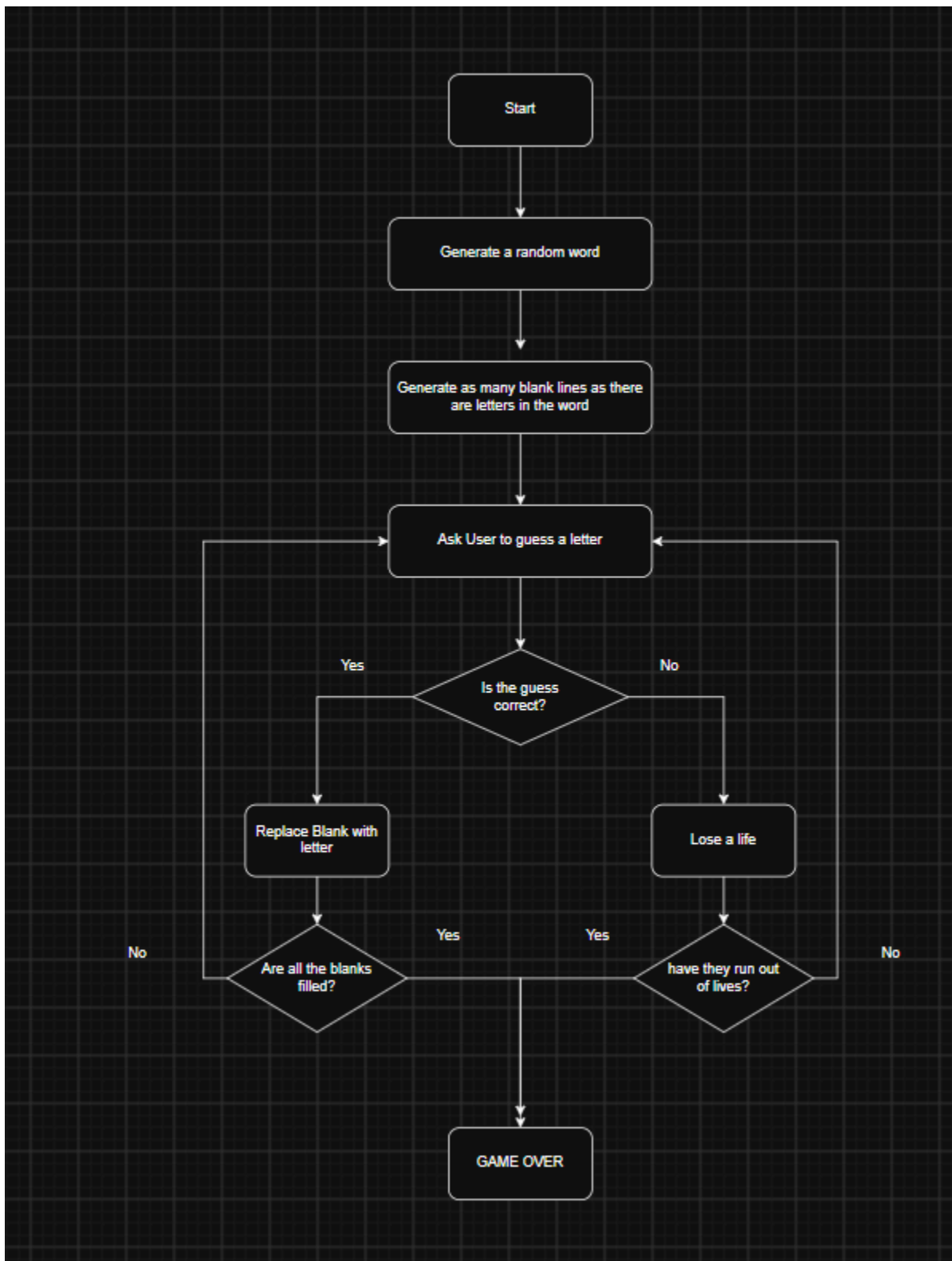


The primary goal for Day 7 was to code Hangman.

The first step in making the hangman game, is to first create a flow chart demonstrating the main parts of the game. Doing this prior to writing any code ensures that core components are understood and in what order they appear.

Figure 1: Flow chart for main parts of hangman



The coding process was broken into 5 steps. Step 1 was to get basic functionality of generating a random word, asking the user to guess a letter, then checking if that letter is in the word.

Figure 2: Code for step 1

```
word_list = ["aardvark", "baboon", "camel"]

# TODO-1 - Randomly choose a word from the word_list and assign it to a variable called chosen_word. Then print it.
import random
random_word = random.choice(word_list)

# TODO-2 - Ask the user to guess a letter and assign their answer to a variable called guess. Make guess lowercase.
guess = input("Guess a letter: ").lower()

# TODO-3 - Check if the letter the user guessed (guess) is one of the letters in the chosen_word. Print "Right" if it
# is, "Wrong" if it's not.

for letter in random_word:
    if letter == guess:
        print("Correct!")
    else:
        print("Guess again.")
```

Step 2 then expanded to adding the visual representation of the random word so the user knows how ,any letters to guess, and if they get it right, where that letter is in the word.

Figure 3: Code for Step 2

```
import random
word_list = ["aardvark", "baboon", "camel"]

chosen_word = random.choice(word_list)
print(chosen_word)

# TODO-1: Create a "placeholder" with the same number of blanks as the chosen_word
placeholder = ''
for letter in chosen_word:
    placeholder += '_'
print (placeholder)
guess = input("Guess a letter: ").lower()

# TODO-2: Create a "display" that puts the guess letter in the right positions and _ in the rest of the string.
display = ''

for letter in chosen_word:
    if letter == guess:
        display += letter
        print("Right")
    else:
        display += '_'
        print("Wrong")

print(display)
```

Figure 4: Output for Step 2

```
camel
-----
Guess a letter: a
Wrong
Right
Wrong
Wrong
Wrong
_a_
```

Step 3 created a while loop so the game continues to run while the win/lose conditions are not met. I then fixed the display variable so that it no longer gets rewritten each iteration of the loop.

Figure 5: Code for Step 3

```
placeholder = ""
word_length = len(chosen_word)
for position in range(word_length):
    placeholder += "_"
print(placeholder)

# TODO-1: - Use a while loop to let the user guess again.

correct_letters = []

while not game_over:
    guess = input("Guess a letter: ").lower()

    display = ""
    # TODO-2: Change the for loop so that you keep the previous correct letters in display.

    for letter in chosen_word:
        if letter == guess:
            display += letter
            correct_letters.append(letter)
        elif letter in correct_letters:
            display += letter
        else:
            display += "_"

    if '_' not in display:
        print("You won!")
        game_over = True
    else:
        game_over = False
```

Stage 4 focused on keeping track of the players life total. The player starts the game with 6 lives. For each incorrect guess, that total is reduced by 1. When the player reaches 0, the game is over. Since this is

hangman, ASCII art of the hangman is updated for each incorrect guess to give the player a visual representation of their life total.

Figure 6: Code for Step 4

```
guess = input("Guess a letter: ").lower()

display = ""

for letter in chosen_word:
    if letter == guess:
        display += letter
        correct_letters.append(guess)
    elif letter in correct_letters:
        display += letter
    else:
        display += "_"

print(display)

# TODO-2: - If guess is not a letter in the chosen_word, Then reduce 'lives' by 1.
# If lives goes down to 0 then the game should stop and it should print "You lose."

if "_" not in display:
    game_over = True
    print("You win.")

if guess not in chosen_word:
    lives -= 1
    print(stages[lives])

if lives == 0:
    game_over = True
# TODO-3: - print the ASCII art from 'stages'
```

```

Guess a letter: n
__m__

  +---+
  |   |
  0   |
 /|\  |
 /    |
      |
=====

Guess a letter: n
__m__

  +---+
  |   |
  0   |
 /|\  |
 / \  |
      |
=====

```

Stage 5 wrapped up the project and improved the user experience. This included importing word list and the ASCII art from separate python files, letting the user know if they already guessed a letter, how many lives they have left, and if they lost, what the correct word was.